

BACK PROPAGATION NEURAL NETWORK

SSK5603 : Lecture Week 5

Back Propagation Neural Network

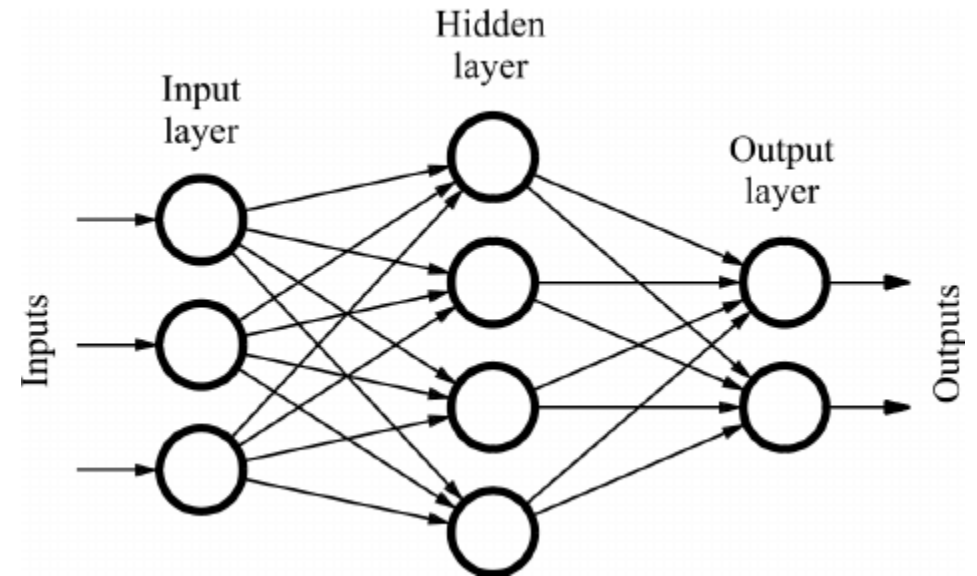
- Backpropagation is the essence of neural network training.
- It is the method of fine-tuning the weights of a neural network based on the error rate obtained in the previous epoch (i.e., iteration).
- Proper tuning of the weights allows you to reduce error rates and make the model reliable by increasing its generalization.
- Backpropagation, short for backward propagation of errors, is a widely used method for calculating derivatives inside deep feedforward neural networks.
- Backpropagation forms an important part of a number of supervised learning algorithms for training feedforward neural networks, such as stochastic gradient descent.

What is Backpropagation Algorithm in Machine Learning?

- When training a neural network by gradient descent, a loss function is calculated, which represents how far the network's predictions are from the true labels.
- Backpropagation allows us to calculate the gradient of the loss function with respect to each of the weights of the network.
- This enables every weight to be updated individually to gradually reduce the loss function over many training iterations.
- Backpropagation involves the calculation of the gradient proceeding backwards through the feedforward network from the last layer through to the first.
- The backpropagation algorithm is key to supervised learning of deep neural networks and has enabled the recent surge in popularity of deep learning algorithms since the early 2000s.

What is a Feed Forward Network?

- A Feed Forward Neural Network is an artificial neural network in which the connections between nodes does not form a cycle.
- The opposite of a feed forward neural network is a recurrent neural network, in which certain pathways are cycled.
- The feed forward model is the simplest form of neural network as information is only processed in one direction.
- While the data may pass through multiple hidden nodes, it always moves in one direction and never backwards.



How does a Feed Forward Neural Network work?

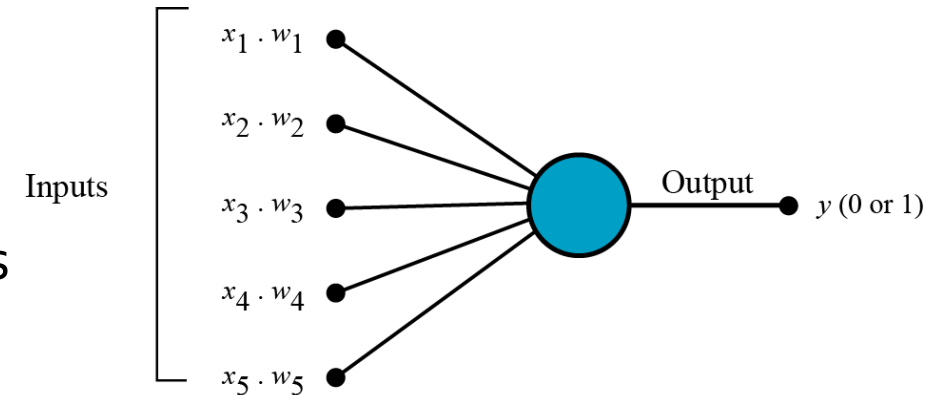
- A Feed Forward Neural Network is commonly seen in its simplest form as a single layer perceptron.
- In this model, a series of inputs enter the layer and are multiplied by the weights.
- Each value is then added together to get a sum of the weighted input values.
- If the sum of the values is above a specific threshold, usually set at zero, the value produced is often 1, whereas if the sum falls below the threshold, the output value is -1.
- The single layer perceptron is an important model of feed forward neural networks and is often used in classification tasks.

How does a Feed Forward Neural Network work?

- Furthermore, single layer perceptrons can incorporate aspects of machine learning.
- Using a property known as the delta rule, the neural network can compare the outputs of its nodes with the intended values, thus allowing the network to adjust its weights through training in order to produce more accurate output values.
- This process of training and learning produces a form of a gradient descent.
- In multi-layered perceptrons, the process of updating weights is nearly analogous, however the process is defined more specifically as backpropagation.
- In such cases, each hidden layer within the network is adjusted according to the output values produced by the final layer.

Applications of Feed Forward Neural Networks

- While Feed Forward Neural Networks are fairly straightforward, their simplified architecture can be used as an advantage in particular machine learning applications.
- For example, one may set up a series of feed forward neural networks with the intention of running them independently from each other, but with a mild intermediary for moderation. Like the human brain, this process relies on many individual neurons in order to handle and process larger tasks.
- As the individual networks perform their tasks independently, the results can be combined at the end to produce a synthesized, and cohesive output.



Types of Backpropagation Networks

Type 1 : Static back-propagation

- It is one kind of backpropagation network which produces a mapping of a static input for static output. It is useful to solve static classification issues like optical character recognition.

Type 2 : Recurrent Backpropagation

- Recurrent Backpropagation in data mining is fed forward until a fixed value is achieved. After that, the error is computed and propagated backward.

The main difference between both of these methods

- The mapping is rapid in static back-propagation while it is non-static in recurrent backpropagation.

Backpropagation Formula

- Let us consider a multilayer feedforward neural network with N layers.
- The output of the first hidden layer is given by

$$\mathbf{h}^{(1)} = g^{(1)}(\mathbf{W}^{(1)\text{T}}\mathbf{x} + \mathbf{b}^{(1)})$$

Feedforward neural network first layer formula

- and the output of the second layer is given by

$$\mathbf{h}^{(2)} = g^{(2)}(\mathbf{W}^{(2)\text{T}}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})$$

Feedforward neural network second layer formula

- And so on, each layer receives the previous layer's output as input. The final layer's output is denoted $\hat{\mathbf{y}}$:

$$\hat{\mathbf{y}}(x) = \mathbf{W}^{(N)\text{T}}\mathbf{h}^{(N-1)} + \mathbf{b}^{(N)}$$

Feedforward neural network last layer formula

Feedforward Neural Network Formula Symbols Explained

$\mathbf{h}^{(i)}$	The output of hidden layer i
$g^{(i)}$	The activation function of hidden layer i , which could be a sigmoid function, a rectified linear unit (ReLU), a tanh function, or similar.
$\mathbf{W}^{(i)}$	The hidden weights matrix in layer i
$\mathbf{x}$	The input vector to the neural network
N	The number of layers in the neural network
$\mathbf{b}^{(i)}$	The bias matrix in layer i

Loss function for backpropagation

- When the feedforward network accepts an input x and passes it through the layers to produce an output $\hat{y}$, information flows forward through the network. This is called forward propagation.
- During supervised learning, the output $\hat{y}$ is compared to the label vector y to give a loss function, also called a cost function, which represents how good the network is at making predictions: $C(y, \hat{y})$

Loss function used for backpropagation

- The loss function returns a low value when the network output is close to the label, and a high value when they are different. So at the start of training, the loss function will be very large, and a fully trained model should have a small loss function, when the training dataset is passed through the network. Examples of loss functions include the cross-entropy loss, the cosine similarity function, and the hinge loss.
- During training, the objective is to reduce the loss function on the training dataset as much as possible. This means that the network weights must gradually be adjusted in order for C to be reduced.

$$\frac{\partial C}{\partial W_{jk}^{(i)}}$$

Loss function used for backpropagation

- This means that we must calculate the derivative of C with respect to every weight in the network:

$$\frac{\partial C}{\partial W_{jk}^{(i)}}$$

- Backpropagation loss function formula symbols explained :

$W_{jk}^{(i)}$	The weight of the network going from node j in layer i - 1 to node k in layer i
----------------	---

Calculating gradients with the chain rule

- Since a neural network has many layers, the derivative of $\mathcal{C}$ at a point in the middle of the network may be very far removed from the loss function, which is calculated after the last layer.
- In fact, $\mathcal{C}$ depends on the weight values via a chain of many functions.
- We can use the chain rule of calculus to calculate its derivative.
- The chain rule tells us that for a function z depending on y , where y depends on x , the derivative of z with respect to x is given by:

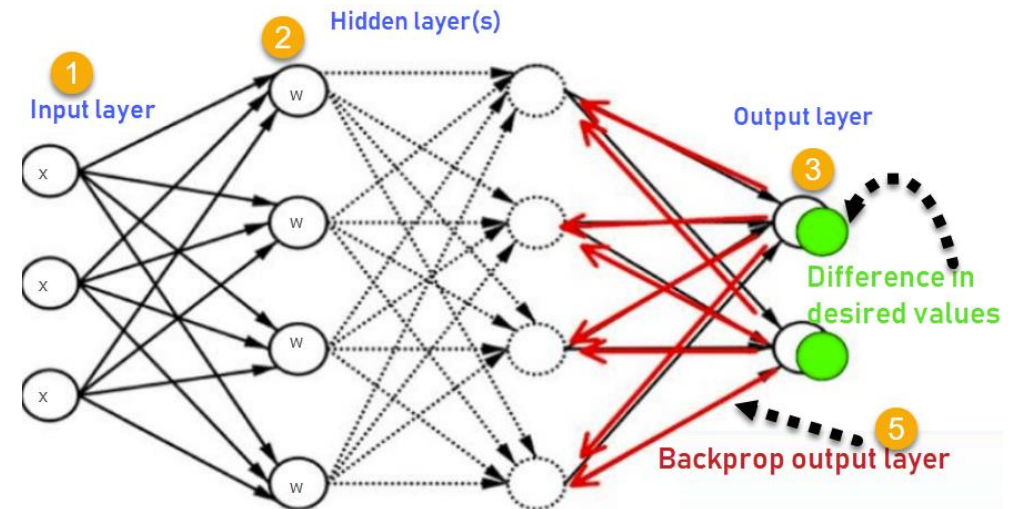
$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

Computing gradients more efficiently with the backpropagation algorithm

- Each component of the derivative of C with respect to each weight in the network can be calculated individually using the chain rule. However, it would be extremely inefficient to do this separately for each weight.
- The backpropagation algorithm involves first calculating the derivatives at layer N , that is the last layer. These derivatives are an ingredient in the chain rule formula for layer $N - 1$, so they can be saved and re-used for the second-to-last layer.
- And so in backpropagation we work our way backwards through the network from the last layer to the first layer, each time using the last derivative calculations via the chain rule to obtain the derivatives for the current layer.
- In this way, the backpropagation algorithm allows us to efficiently calculate the gradient with respect to each weight by avoiding duplicate calculations.

How Backpropagation Algorithm Works

- The Back propagation algorithm in neural network computes the gradient of the loss function for a single weight by the chain rule.
- It efficiently computes one layer at a time, unlike a native direct computation.
- It computes the gradient, but it does not define how the gradient is used.
- It generalizes the computation in the delta rule.
- Consider the backpropagation neural network example diagram to understand →



How Backpropagation Algorithm Works

1. Inputs X , arrive through the pre-connected path
2. Input is modeled using real weights W . The weights are usually randomly selected.
3. Calculate the output for every neuron from the input layer, to the hidden layers, to the output layer.
4. Calculate the error in the outputs

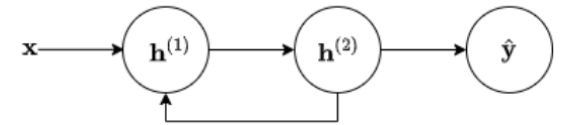
$$\text{Error}B = \text{Actual Output} - \text{Desired Output}$$
5. Travel back from the output layer to the hidden layer to adjust the weights such that the error is decreased.

Keep repeating the process until the desired output is achieved

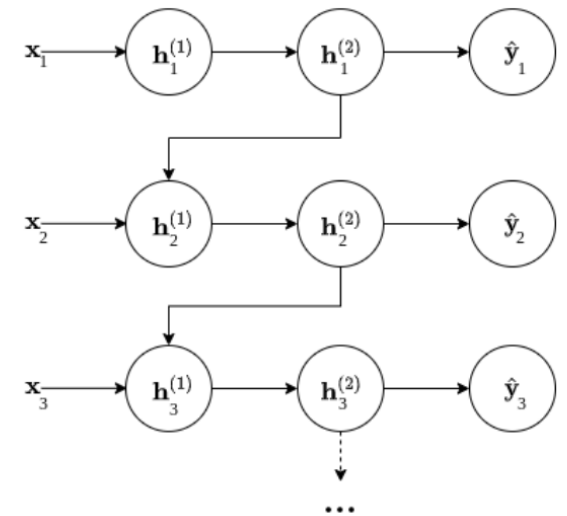
Standard Backpropagation vs. Backpropagation Through Time

- Backpropagation relies on the ability to express an entire neural network as a function of a function of a function... and so on, allowing the chain rule to be applied recursively.
- This cannot be applied if the neural network cannot be reduced to a single expression of compounded functions - in other words, if it cannot be expressed as a directed acyclic graph.
- A recurrent neural network processes an incoming time series, and the output of a node at one point in time is fed back into the network at the following time point. This means that a recurrent neural network cannot be expressed as a directed acyclic graph, since it contains cycles.

Recurrent neural network:



Unrolled for backpropagation through time:



Standard Backpropagation vs. Backpropagation Through Time

- However, it is possible to 'unroll' a recurrent neural network and view it like a feedforward neural network. Each timestep is represented as a single copy of the original neural network.
- Unrolling a recurrent neural network in order to represent it as a feedforward neural network for backpropagation through time
- Because backpropagation through time involves duplicating the network, it can produce a large feedforward neural network which is hard to train, with many opportunities for the backpropagation algorithm to get stuck in local optima.
- Furthermore, interactions between inputs that are far apart in time can be hard for the network to learn, as the gradient contributions from the interaction become diminishingly small in comparison to local effects. This is known as the vanishing gradient problem, and can be addressed by choosing ReLU activation functions, and introducing regularization into the network.

Why We Need Backpropagation?

Most prominent advantages of Backpropagation are:

- Backpropagation is fast, simple and easy to program
- It has no parameters to tune apart from the numbers of input
- It is a flexible method as it does not require prior knowledge about the network
- It is a standard method that generally works well
- It does not need any special mention of the features of the function to be learned.

Applications of Backpropagation

Backpropagation are widely used for training nearly all kinds of neural networks

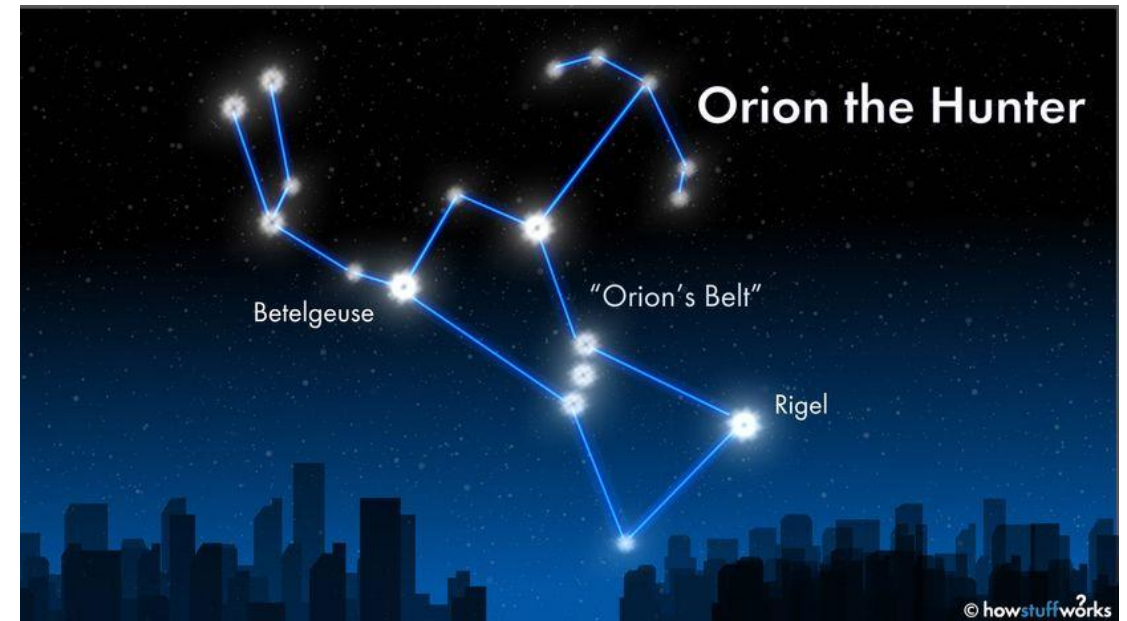
Have enabled the recent surge in popularity of deep learning

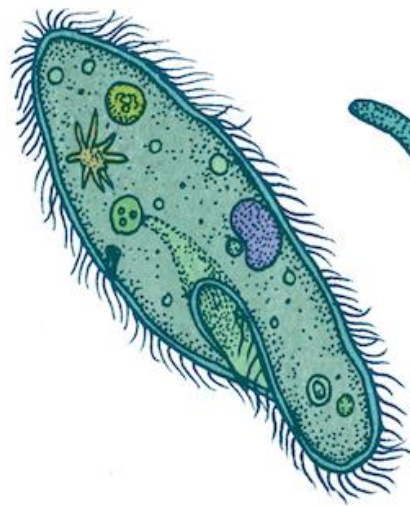
Pattern Recognition

- An important application of neural networks is pattern recognition. Pattern recognition can be implemented by using a feed-forward neural network that has been trained accordingly.
- During training, the network is trained to associate outputs with input patterns.
- When the network is used, it identifies the input pattern and tries to output the associated output pattern.
- The power of neural networks comes to life when a pattern that has no output associated with it, is given as an input.
- In this case, the network gives the output that corresponds to a taught input pattern that is least different from the given pattern.

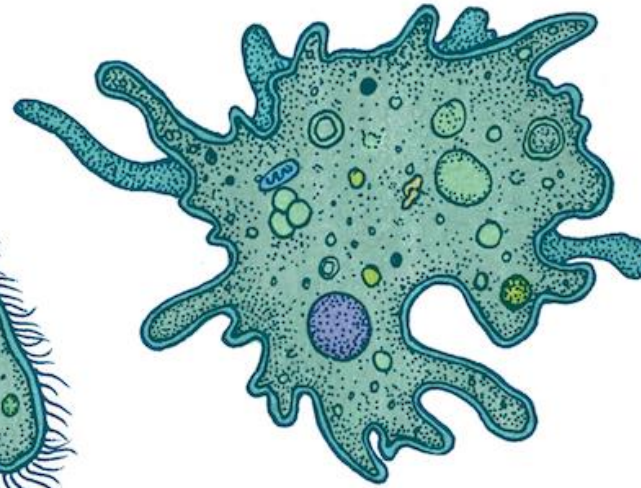
Pattern Recognition

- Pattern
 - a set of objects or phenomenon or concepts where the elements of the set are similar to one another in certain ways/as[pects. The pattern are described by certain quantities, qualities, traits, notable features and so on.
- Example
 - Pattern of constellations
- Human perception has strong tendency to find patterns from anything. We see patterns from even random noise – we are more likely to believe a hidden pattern than denying it when the risk (reward) for missing (discovering) a pattern is often high.

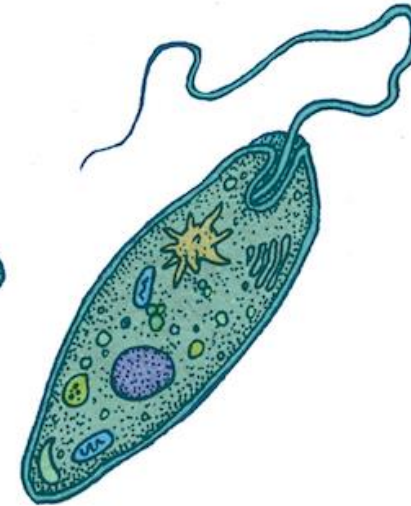




Paramecium



Amoeba



Euglena

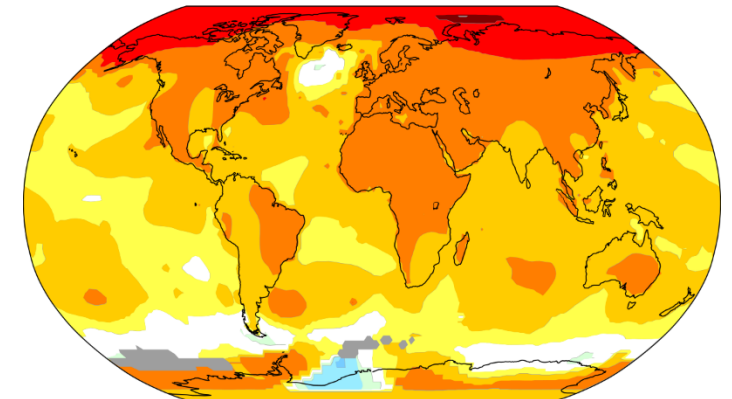


18/5/2026

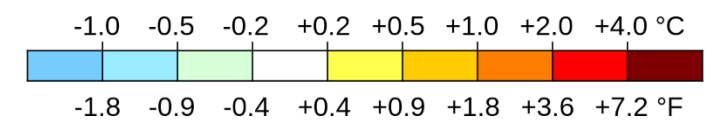


Maslina @ SSK5603

Temperature change in the last 50 years

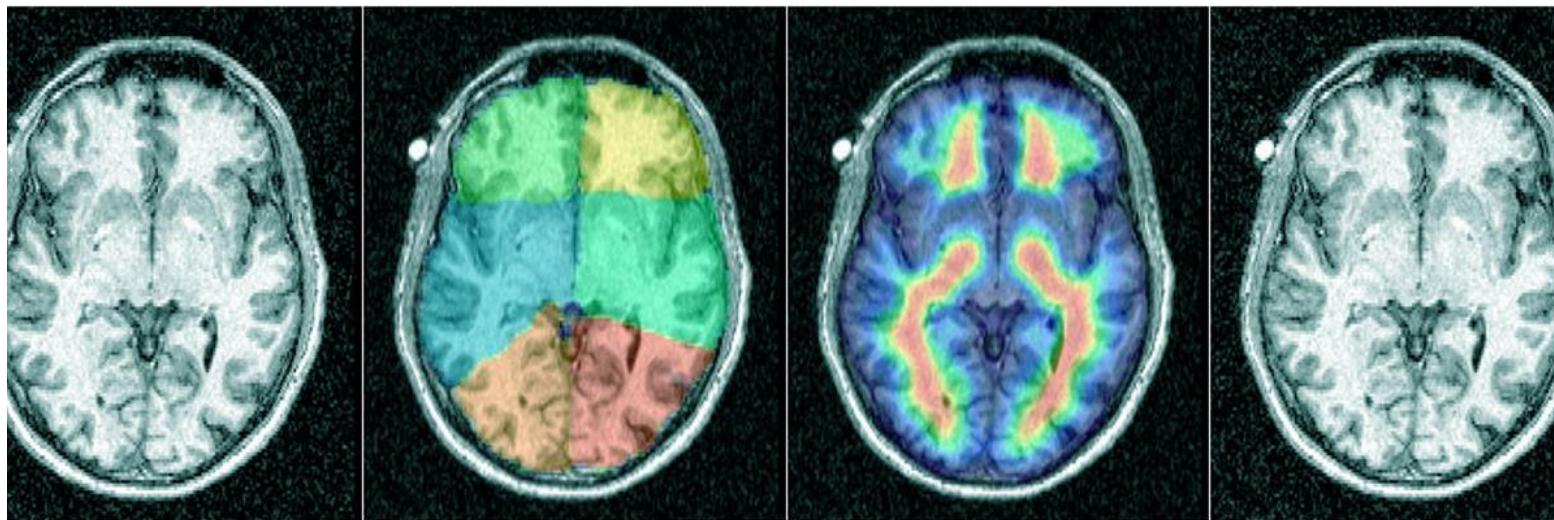


2011-2021 average vs 1956-1976 baseline





Optical Character Recognition



Backpropagation in convolutional neural networks for face recognition.

- Convolutional neural networks are the standard deep learning technique for image processing and image recognition, and are often trained with the backpropagation algorithm.
- In 2015, Parkhi, Vidal and Zisserman described a technique for building a face recognizer. They used a convolutional neural network with 18 layers, and a database of celebrity faces.
- Initially, the network was trained using backpropagation through all the 18 layers. Images were passed into the network in batches, the loss function was calculated, and the gradients were calculated first for layer 18, working back towards layer 1. After each batch of images, the network weights were updated.
- To refine the network to be able to distinguish the nuances of human faces, the researchers ran an extra training stage for layer 18 only, once the backpropagation had run for all 18 layers. The researchers chose a loss function called a triplet loss.
- The neural network receives an input of three celebrity face images at once, for example, two images of Matt Damon and one image of Brad Pitt.
- The loss function penalizes the network if it decides that two images of the same person are different, and also penalizes the network for classifying images of different people as similar. Over time, triplets of three images are passed through the network and the loss function is calculated, and the weights of the last layer are updated.

Backpropagation for speech recognition

- Speech recognition system for English and Japanese
- able to run on embedded devices
- developed by the Sony Corporation of Japan.
- The system is designed to listen for a limited number of commands by a user.
- Implementation:
 - an incoming sound signal is split into windows of time, and a Fast Fourier Transform is applied.
 - The sound intensity at different frequencies is taken as a feature and input into a neural network consisting of five layers.
 - The researchers chose a softmax cross-entropy loss function, and were able to apply backpropagation to train the five layers to understand Japanese commands.
 - They were then able to switch the network to train on English sound recordings, and were able to adapt the system to recognize commands in English.
- This is an example of transfer learning: a machine learning model can be trained for one task, and then re-trained and adapted for a new task.

Backpropagation History

1847

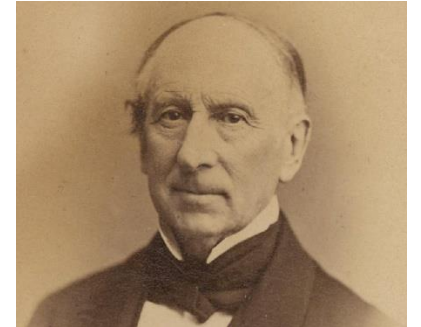
- the French mathematician Baron Augustin-Louis Cauchy
- developed a method of gradient descent for solving simultaneous equations
- had the idea of taking the derivative of a function and taking small steps to minimize an error term.

Over the following century

- gradient descent methods were used across disciplines to solve difficult problems numerically
- where an exact algebraic solution would have been impossible or computationally intractable.

1970

- the Finnish master's student Seppo Linnainmaa described an efficient algorithm for error backpropagation in sparsely connected networks in his master's thesis at the University of Helsinki
- although he did not refer to neural networks specifically.



Augustin-Louis Cauchy
(1789-1857)
Inventor of gradient
descent.

Backpropagation History

1986

- the American psychologist David Rumelhart and his colleagues published an influential paper applying Linnainmaa's backpropagation algorithm to multi-layer neural networks.
- The following years saw several breakthroughs building on the new algorithm, such as Yann LeCun's 1989 paper applying backpropagation in convolutional neural networks for handwritten digit recognition.

1980s

- various researchers independently derived backpropagation through time, in order to enable training of recurrent neural networks.

Recent years

- Deep neural networks have become ubiquitous and backpropagation is very important for efficient training.
- Although the algorithm has been modified to be parallelized and run easily on multiple GPUs, Linnainmaa and Rumelhart's original backpropagation algorithm forms the backbone of all deep learning-based AI today.

Backpropagation Key Points

- Simplifies the network structure by elements weighted links that have the least effect on the trained network
- You need to study a group of input and activation values to develop the relationship between the input and hidden unit layers.
- It helps to assess the impact that a given input variable has on a network output. The knowledge gained from this analysis should be represented in rules.
- Backpropagation is especially useful for deep neural networks working on error-prone projects, such as image or speech recognition.
- Backpropagation takes advantage of the chain and power rules allows backpropagation to function with any number of outputs.

Best practice Backpropagation

- Backpropagation in neural network can be explained with the help of “Shoe Lace” analogy

Too little tension =

- Not enough constraining and very loose

Too much tension =

- Too much constraint (overtraining)
- Taking too much time (relatively slow process)
- Higher likelihood of breaking

Pulling one lace more than other =

- Discomfort (bias)

Disadvantages of using Backpropagation

- The actual performance of backpropagation on a specific problem is dependent on the input data.
- Back propagation algorithm in data mining can be quite sensitive to noisy data.
- You need to use the matrix-based approach for backpropagation instead of mini-batch.

References

- <https://deepai.org/machine-learning-glossary-and-terms/backpropagation>
- <https://www.guru99.com/backpropagation-neural-network.html>